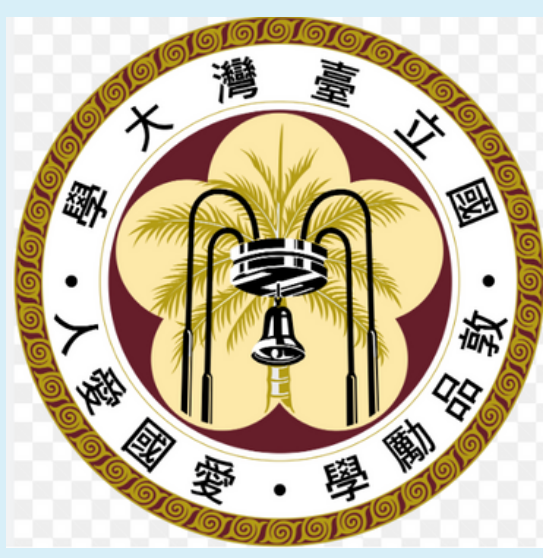
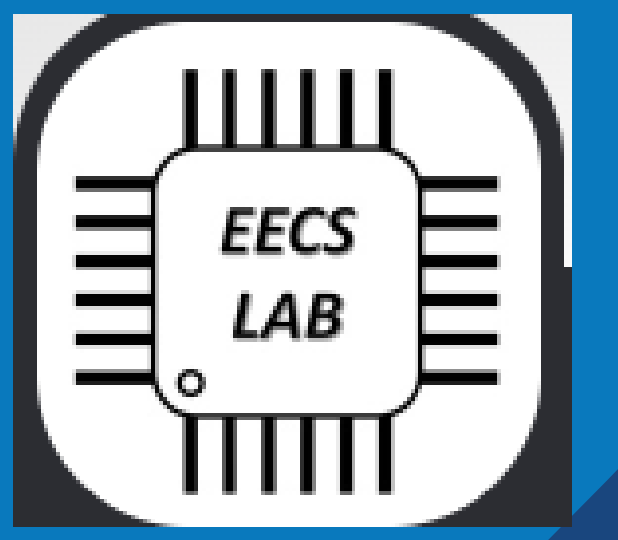




QUANTIZED BERT ON CIM ACCELERATORS

Authors: Jason Cheng¹, Bing-xuan Huang², Cindy Liu², Tsung-te Liu³

Affiliations: National Taiwan University, College of Electrical Engineering and Computer Science (EECS Lab)



Introduction

iBERT is a lightweight version of BERT (Bidirectional Encoder Representations from Transformers) designed to run efficiently using only integer operations instead of floating-point math. This makes it better suited for deployment on hardware with limited resources. It replaces standard operations like softmax and normalization with quantized versions that are faster and more hardware-friendly. In our project, we trained and quantized a smaller BERT model using quantization-aware techniques (QAT). While we didn't fully deploy it on the hardware accelerator, we prepared the outputs and lookup tables to be compatible for future integration.

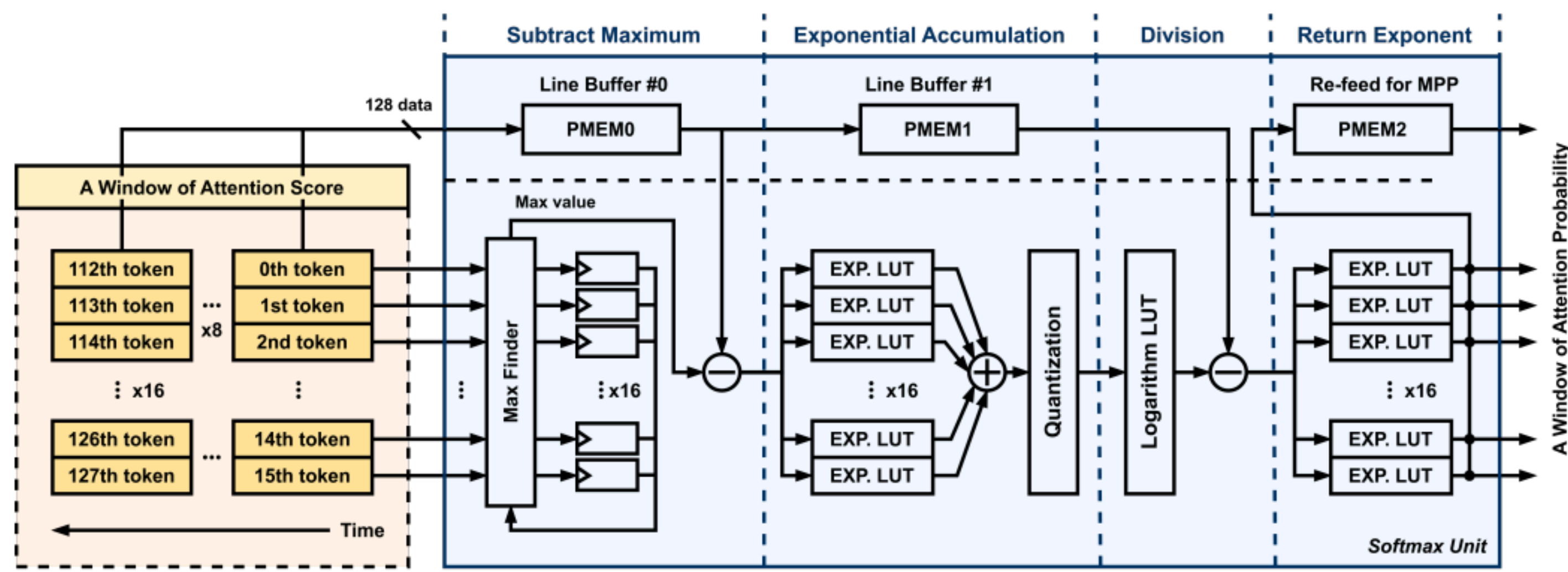


Figure 4.26: The implementation of PSC.

PSC (Pipelined Softmax Core) used for quantized attention in iBERT deployment. The softmax unit replaces traditional floating-point division with log-based LUTs to enable efficient INT8 computation. Our QAT-trained model outputs were formatted to integrate with this module.

Quantization

To prepare our model for compatibility with the CIM-based transformer accelerator, we applied quantization-aware training (QAT) and adjusted it to match the chip's hardware limits

- Replaced floating-point operations with integer-friendly alternatives using lookup tables (LUTs). This included rewriting softmax to use exponential and logarithmic approximations compatible with the PSC module.
- Trained with INT8 quantization for weights and activations across all layers, preserving accuracy while enabling efficient inference on the chip.
- Generated softmax LUTs separately via a Python script, matching the bit-serial architecture and streaming requirements of the hardware softmax core.

Our model also uses a reduced embedding size of 512 (vs. 768 in BERT-base), to save memory and speed up computation, while still keeping good performance.

$$\begin{aligned} \text{Softmax}(x_i) &= \frac{\exp(x_i - x_{\max})}{\sum_{j=1}^N \exp(x_j - x_{\max})} \\ &= \left\{ \exp \left[(x_i - x_{\max}) - \log \left(\sum_{j=1}^N \exp(x_j - x_{\max}) \right) \right] + S_{\text{bias}} \right\} \gg S_{\text{shift}} \end{aligned}$$

Data

Full_GLUE_QAT_Results

Task	Metric	INT8 (QAT)
CoLA	Matthews Correlation	0.48
MNLI-m	Accuracy	0.801
MNLI-mm	Accuracy	0.806
MRPC	F1	0.893
QNLI	Accuracy	0.875
QQP	F1	0.866
RTE	Accuracy	0.567
SST-2	Accuracy	0.9

Results from the INT8 quantized BERT model on GLUE tasks. Each task uses its main evaluation metric. The model keeps good performance even after quantization.

Methodology/Implementation

We began with a smaller BERT model from Huggingface's Transformers library and trained it using PyTorch. After fine-tuning on the GLUE benchmark (a collection of language understanding tasks used to evaluate the performance of NLP models.), we applied quantization-aware training (QAT) to make the model more efficient for deployment. Once training was complete, we exported the model's weights and biases as JSON files and generated the required lookup tables (LUTs) using a Python script. These outputs were formatted to match the CIM-based accelerator's requirements for future integration.

Transformer Accelerator

The accelerator we prepared our model for is a CIM-based transformer chip developed by Yi-Hsuan Pan. It's designed for efficient large language model inference by performing matrix multiplications directly inside SRAM (Static Random-Access Memory) arrays, which helps reduce memory access overhead.

This accelerator includes several features that work well with our quantized model:

- PSC-based Softmax:** The pipelined softmax core (PSC) uses log-based exponential and division approximations through lookup tables (LUTs). We generated these tables during QAT to integrate with this unit.
- INT8-Compatible CIM Cores:** The chip's static and dynamic CIM arrays are optimized for 8-bit integer operations, making them well-suited for models trained with quantization-aware techniques.
- Fully-Pipelined Architecture:** Data flows through the accelerator in a streaming fashion, which reduces the need for intermediate storage and improves overall processing speed.

The model outputs, including weights and LUTs, were formatted to be compatible with this pipeline for future integration.

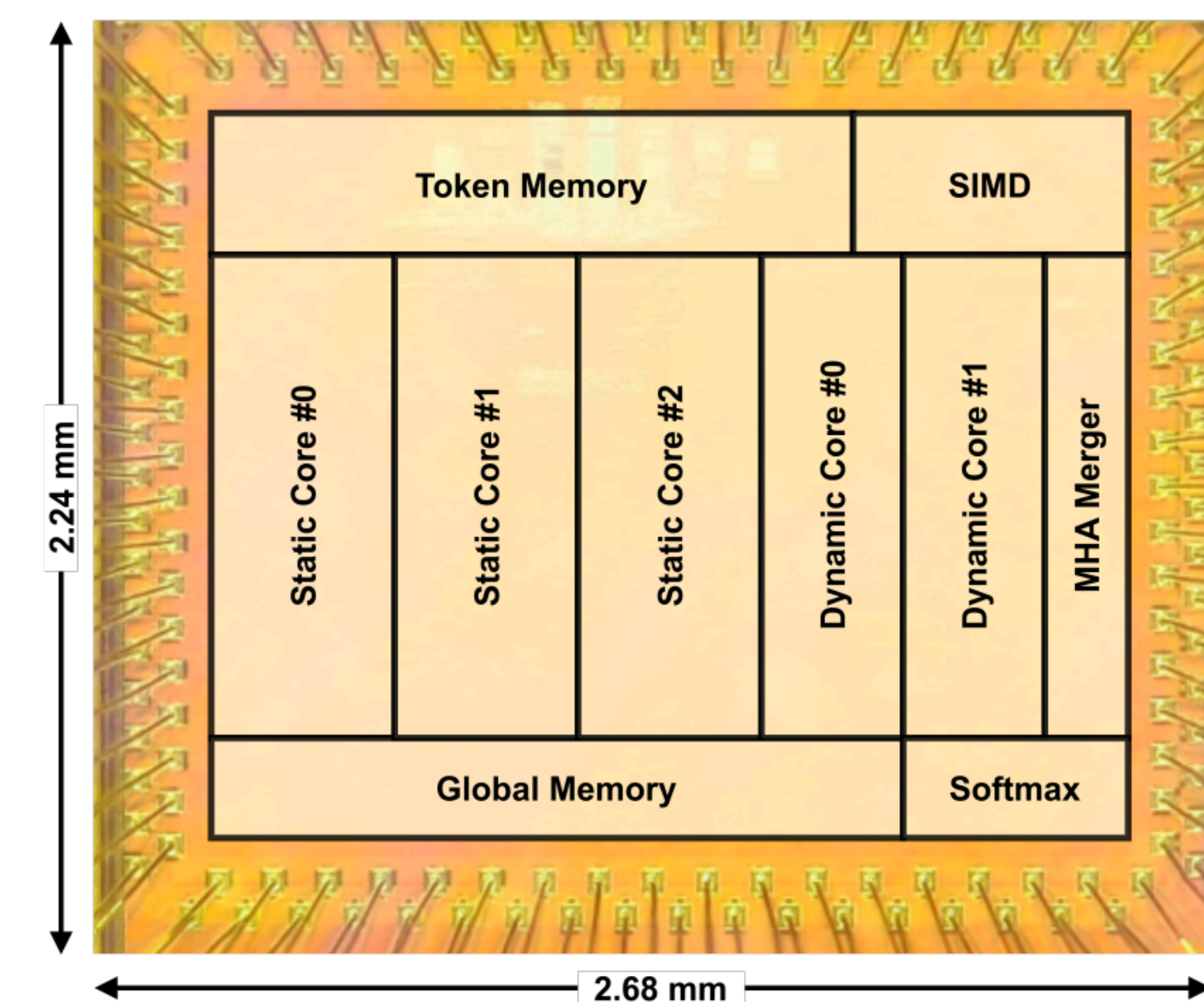


Figure 5.1: Die photo of the proposed Transformer accelerator.

Fabricated in 28nm CMOS (Complementary Metal-Oxide-Semiconductor), this chip integrates PSC, static and dynamic CIM cores, and supports efficient quantized transformer inference.

Conclusion

Through this project, I gained hands-on experience in preparing transformer models for real-world hardware deployment. Working with quantization-aware training (QAT) taught me how to adapt deep learning models to meet hardware constraints without sacrificing performance. I learned how to bridge the gap between software and hardware by formatting model outputs and lookup tables to align with a custom compute-in-memory system. This gave me a deeper understanding of hardware-aware model design and the importance of co-design in AI systems.

References

- Devlin et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers. NAACL-HLT.
- Kim et al. (2021). I-BERT: Integer-only BERT Quantization. ICML.
- Vaswani et al. (2017). Attention Is All You Need. NeurIPS.
- Han et al. (2024). Efficient AI Systems. MIT 6.S940: Hardware-Aware Efficient AI Systems.